

NeurIPS 2023 | Compressing a Matrix to One Bit per Entry

Stanford's LPLR fuses randomized low-rank factorization with low-precision quantization, matching or beating standard compression on images, embeddings, and LLaMa weights

Matrices are the workhorse data structure of science and machine learning, but modern ones can hold billions of entries. Simply storing them, moving them across memory, and multiplying them strains both memory and compute. What rescues us is a fact that is easy to overlook: many real-world matrices are approximately low rank. They carry far less information than their raw dimensions suggest.

There are two well-worn ways to exploit that redundancy. One is low-rank approximation, which writes a matrix as the product of a tall factor and a wide factor, collapsing the number of entries. The other is low-precision quantization, which stores each number in a handful of bits instead of a 32-bit float. Both are mature on their own. Yet folding them into a single factorization, with a provable error guarantee, has remained an open problem.

At NeurIPS 2023, a team from Stanford proposes LPLR (Low-Precision Low-Rank factorization): a single randomized routine that produces two factors that are simultaneously low rank and low precision, so that $A \approx LR$, and that comes with upper bounds on the approximation error as a function of the target rank and the bit budget. Across a deliberately varied set of matrices — from medical test images to embedding vectors to the weight matrices of LLaMa-7b — LPLR compresses down to roughly one bit per entry while matching, and in some settings exceeding, the accuracy of conventional compression.

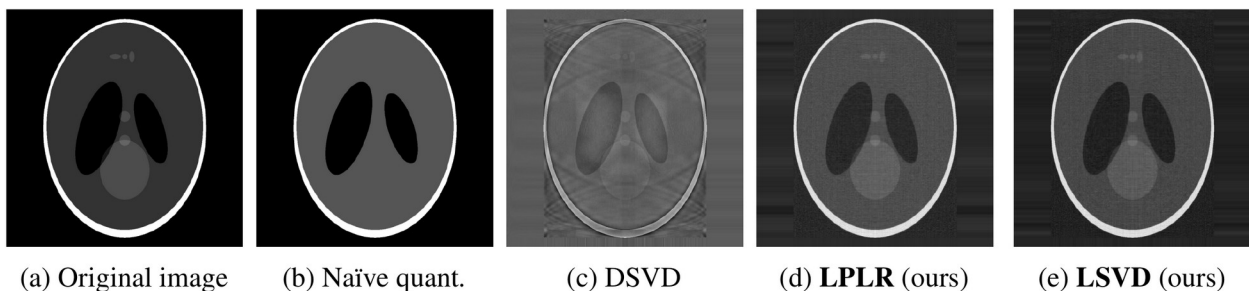


Figure 1. Compressing the Shepp-Logan phantom, a standard medical-imaging test image, at a fixed total budget of about $2 \cdot 10^6$ bits. (a) the original; (b) naive quantization at 2 bits per pixel washes out the internal structure; (c) quantizing the SVD factors directly (DSVD) introduces visible artifacts; (d) LPLR and (e) LSVD (this work) factor the image into a product of tall-and-wide matrices, which lets each stored entry use more precision and best preserves fine detail such as the small internal ellipses.

Paper: <https://arxiv.org/abs/2310.11028>

Code: <https://github.com/pilancilab/matrix-compressor>

Why combining the two is hard

The intuition is simple enough. If a matrix is low rank, we can write it as a tall factor times a wide factor, which slashes the entry count. If we then store those factors in low precision, we save even more. It looks like a matter of stacking the two steps. In practice, two obstacles get in the way.

The first is accuracy. Combining low-rank factorization and quantization crudely — especially at aggressive bit budgets — lets error accumulate quickly, and quality collapses. The second is cost. The most accurate route to low-rank factors is the singular value decomposition (SVD), but its $O(nd^2)$ cost is prohibitive for the largest matrices on today's GPU hardware. The paper asks whether there is a fast, analyzable, randomized path that yields low-rank and low-precision factors in one shot while keeping the approximation error under firm control.

How LPLR works

LPLR runs in a few steps. It first draws a Gaussian random matrix S and forms the sketch AS — a randomized rangefinder that approximates the column space of A . It quantizes this basis to obtain $Q(AS)$, which becomes the first factor, L .

Next it projects the columns of A onto the quantized basis by solving the least-squares problem $W^* = \operatorname{argmin}_W \|Q(AS) \cdot W - A\|_F^2$, which has a closed form and can also be solved iteratively with conjugate gradients. Finally it quantizes W^* with a second quantizer, returning the two factors $L = Q(AS)$ and $R = Q'(W^*)$. The whole procedure costs $O(ndm)$ with sketch size m — far less than the $O(nd^2)$ of a direct SVD.

The paper backs this with an error bound: $\|LR - A\|_F^2 \leq (1 + k/(m-k-1)) \cdot \|A_k - A\|_F^2 + \epsilon$, where A_k is the best rank- k approximation of A , k is the target rank, and ϵ is set by the quantization budget. The bound cleanly separates the truncation error from choosing a low rank and the precision error from quantization, giving practitioners an explicit knob to trade compression ratio against accuracy.

Why one bit still holds up

LPLR stays stable at such aggressive budgets because of a property of the Gaussian sketch. A Gaussian matrix is a Johnson-Lindenstrauss (JL) embedding, and it brings an equalization effect: the quantization error of the reconstruction $\hat{x} = S^T Q(Sx)$ stays $O(1)$ rather than growing to $O(d)$ with the dimension, as entrywise quantization would. That is precisely what keeps LPLR accurate even when each coordinate is stored with a single sign bit. It also explains a counterintuitive observation in the paper — spending bits on thinner factors often pays off more than piling bits onto the original matrix.

Evidence 1: downstream classification of embeddings

The first experiment asks whether the compressed matrix is still usable. The authors generate CIFAR-10 and CIFAR-100 image embeddings with MobileNetV3 and IMDB and Emotion text embeddings with BERT, compress the embedding matrices, and then measure downstream accuracy and weighted F1 with a k -nearest-neighbor classifier. Every method is

compared at parity — the same total bit budget — where B_{nq} denotes the bits per entry used by the naive-quantization baseline.

Table 1. Embedding classification accuracy (%) at a baseline budget of $B_{nq} = 1$ bit, with $B = B' = 8$ bits for the LPLR-family methods. Naive quantization all but collapses at one bit, whereas LPLR and LSVD match or exceed the unquantized baseline (kNN benefits from the denoising).

Dataset	LPLR	LPLR-SVD	DSVD	Naive Quant.	Unquantized
CIFAR-10	92	92	92	11	91
CIFAR-100	79	82	82	1	76

The contrast is stark. At the extreme setting $B_{nq} = 1$, naive quantization falls to 11% accuracy on CIFAR-10 and to roughly 1% on CIFAR-100 — essentially random guessing — while LPLR holds 92% and 79%, on par with or above the unquantized 91% and 76%. In other words, LPLR compresses the embeddings to about one bit per entry with almost no loss in task performance.

Evidence 2: compressing LLaMa-7b weights

Large language models make the point more forcefully. The authors compress the weight matrices of LLaMa-7b layer by layer and measure reconstruction quality with the relative Frobenius norm error — the lower the error, the closer the compressed weights are to the originals. The figure below plots the per-layer error for LPLR, LPLR-SVD, and naive quantization in the original layer order.

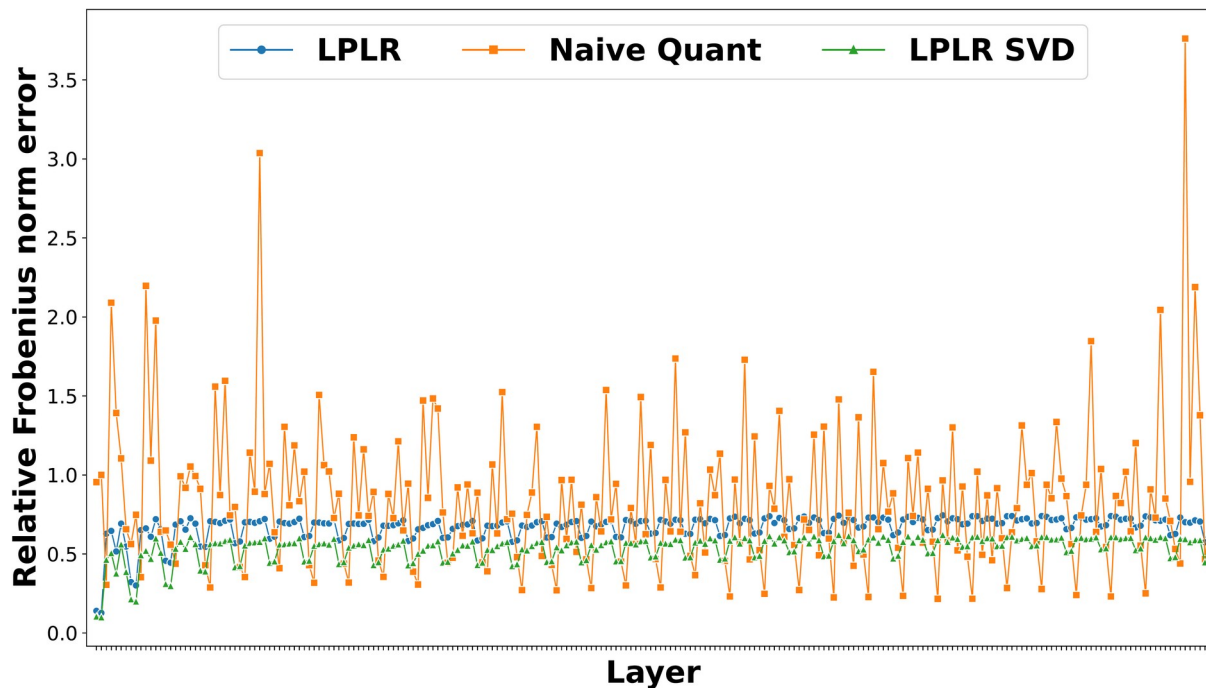


Figure 2. Relative Frobenius norm error on the LLaMa-7b weight matrices for LPLR (blue), LPLR-SVD (green), and naive quantization (orange), ordered by the original sequence of layers. Naive quantization is high and volatile across most layers, spiking above 1.5 repeatedly, while LPLR and LPLR-SVD stay consistently lower and far steadier.

The orange naive-quantization curve swings wildly, spiking above 1.5 and even past 3.0 on many layers, while the blue LPLR and green LPLR-SVD traces hug a lower, flatter band. Aggregated into numbers, the same advantage holds.

Table 2. Mean relative Frobenius norm error on the LLaMa-7b weight matrices (lower is better). Under both bit budgets, LPLR-SVD sits well below naive quantization, with far smaller layer-to-layer variance.

Bit budget	LPLR	LPLR-SVD	Naive Quant.
$B = B' = 8, B_{nq} = 4$	0.672	0.537	0.836
$B = B' = B_{nq} = 4$	0.548	0.540	0.836

At $B = B' = 8$ and $B_{nq} = 4$, LPLR-SVD reaches a mean relative error of 0.537 against 0.836 for naive quantization. Not only is the mean lower; the standard deviation is much smaller too (0.079 versus 0.470), meaning its behavior across layers is more stable and more predictable.

Teasing apart low rank and low precision

Does LPLR's gain come from the low-rank structure or from the quantization precision? The paper separates the two with an ablation. Holding the compression ratio fixed, the authors sweep the bit-budget triplet (B, B', B_{nq}), and find that shifting budget away from the low-rank factors (B) toward the non-quantized reference (raising B_{nq}) systematically lowers the Frobenius error for LPLR and its SVD variant. Across the embedding experiments, raising B_{nq} from 1 to 4 bits moves LPLR from decisively beating naive and Direct-SVD quantization at the aggressive end toward parity with them at the relaxed end. The reading is clear: the more aggressive the compression, the larger LPLR's edge.

Takeaway and boundaries

The message of this work is that low rank and low precision are complementary, and exploiting them jointly is what pushes compression to the extreme. With one Gaussian random projection and two rounds of quantization, LPLR compresses a matrix to roughly one bit per entry while preserving accuracy on images, embeddings, and LLM weights — and it carries provable upper bounds on the approximation error. Because it sidesteps the expensive SVD and runs in $O(ndm)$ time, it is especially practical for the very largest matrices where exact methods simply do not fit.

It is worth being precise about the boundary. LPLR's guarantee rests on the matrix being approximately low rank; for matrices that are genuinely high rank, with slowly decaying singular values, low-rank factorization has little to save in the first place. The paper also observes that as B_{nq} relaxes toward 4 bits, LPLR's advantage over naive quantization narrows to a near tie — its brightest regime is exactly the aggressive end where competing methods break down first.